Trabajo Práctico N.º 8 · Food Store en la feria

La base de la tablet y el cierre de la capa de datos · Base de Datos II · Tecnicatura Universitaria en Programación · UTN

> **Presentado por:** Moran, Joaquín Leandro · Este informe cubre 1.a, 2.c, 3.a, 3.b y 3.c. Los scripts, el orden para crear todo desde cero y la salida de cada paso están en el `README.md` y en `evidencias/`. Regla de oro: **el móvil propone, el servidor dispone.**
> IA: la spec se revisó en **Kiro**; el esquema y el semáforo los generó **OpenCode** (Build) a partir de las tareas, y el code review lo hizo OpenCode (Plan); la primera versión y los scripts de apoyo, con **Claude**. El detalle está en `docs/DUIA.md` y `docs/bitacora_ia.md`. Todo se corrió en PostgreSQL 18.6 y SQLite 3.54.1.

1.a Qué datos viajan

En el servidor (PostgreSQL)	En la tablet (SQLite)	¿Viaja? Por qué
`producto.id`, `producto.nombre`	`producto_cache.id`, `nombre`	Sí. Identifican el producto en el catálogo y en los borradores.
`producto.precio NUMERIC(10,2)`	`precio_centavos INTEGER`	Sí, en centavos (1050,00 → 105000): se compara sin errores de redondeo. La vista lo calcula como `BIGINT` para que un precio alto no desborde (#13).
`producto.stock`	`stock_visto`	Sí, como referencia: es el stock del momento de la copia, no el real.
`producto.activo` + `categoria.activo` + stock > 0	`disponible` (0/1)	Sí, calculado en `v_snapshot_movil`: si se puede ofrecer ahora.
`NOT producto.activo` (baja lógica)	`eliminado` (0/1)	Sí: es la lápida. La tablet se entera de la baja y conserva la fila.
`now()` al exportar	`copiado_en`	Sí: la fecha de la copia, para saber qué tan vieja es.
`COUNT(*) OVER ()` al exportar	— (`filas_totales`, solo en el CSV)	Sí, como control: la carga exige que lleguen todas las filas antes de tocar el catálogo; un CSV cortado no deja lápidas falsas (#4).
`descripcion`, `categoria_id`, `created_at`	—	No: la tablet no los usa para mostrar el catálogo ni para armar un borrador.
`cliente` (nombre, apellido, email, teléfono) y usuarios con sus credenciales	—	No: son datos personales y una tablet en una feria se puede perder. Quien carga el pedido se guarda como un código (`feria-01`); un `CHECK` rechaza cualquier texto con `@`.
`pedido`, `detalle_pedido`	—	No: son ventas, y las ventas las decide el servidor.

Viaja solo lo necesario para mostrar el catálogo y armar borradores sin señal. **La fuente de verdad es PostgreSQL:** la tablet tiene una copia con fecha que puede estar vieja, por eso cada ítem guarda el precio que vio el cliente y el semáforo lo compara con una instantánea nueva antes de dar nada por bueno.

2.c Los casos

Cada caso sigue el mismo ciclo: **borrador en la tablet → cambio en la copia del servidor (si corresponde) → instantánea nueva → `cargar_snapshot.sql` → `semaforo.sql`**. La instantánea se exporta con `\copy (SELECT * FROM v_snapshot_movil ORDER BY id) TO 'movil/servidor_snapshot.csv' CSV HEADER`. Todo lo corre `ejecutar_todo.sh` sobre la copia `foodstore_tp8`; la salida completa está en `evidencias/2c_C2.txt` a `2c_C6.txt`.

C2 · Sube el precio después del borrador → `CONFLICTO`, el cliente acepta → `SIMULADO_APROBADO`

```
$ sqlite3 ... < casos/c2_borrador.sql                  -- 2 Muzzarella a 1050.00 y 1 Coca 1.5L
$ psql -d foodstore_tp8 -f servidor/casos/c2_subir_precio.sql
  UPDATE producto SET precio = 1150.00 WHERE nombre = 'Muzzarella';
$ (instantánea nueva) · cargar_snapshot.sql · semaforo.sql
| a001c001 | b001c000 | SIMULADO_APROBADO | coincide con la instantánea (simulación: no es una venta) |
| a0c20001 | b0c20000 | CONFLICTO         | Muzzarella: precio visto 1050.00, precio actual 1150.00   |
$ sqlite3 ... < casos/c2_acepta.sql     -- precio visto = el de la copia nueva; solicitud NUEVA
$ sqlite3 -bail ... < semaforo.sql
| a0c20002 | b0c20000 | SIMULADO_APROBADO | coincide con la instantánea (simulación: no es una venta) |
```

No hay rojo (el producto existe, está disponible y hay stock), pero el precio visto difiere del actual: amarillo. Cuando el cliente acepta, se actualiza el precio del borrador y se manda otra solicitud con otro UUID; la primera queda en `CONFLICTO` como historial. En esa misma corrida salió en verde la solicitud de la prueba 1.c, que estaba `PENDIENTE`.

C3 · Borrador por más unidades de las que quedan → `RECHAZADO`

```
$ sqlite3 ... < casos/c3_borrador.sql      -- 20 Flan casero; la copia dice stock_visto = 15
$ (instantánea nueva) · cargar_snapshot.sql · semaforo.sql
| a0c30001 | b0c30000 | RECHAZADO | Flan casero: pide 20 y quedan 15 |
```

La tablet no bloquea el borrador, porque solo propone. El semáforo compara la cantidad con el stock de la instantánea y da rojo. No hizo falta tocar el stock del servidor.

C4 · Producto dado de baja después del borrador → `RECHAZADO`, la fila queda

```
$ sqlite3 ... < casos/c4_borrador.sql                  -- 1 Fugazzeta y 3 Empanadas de pollo
$ psql -d foodstore_tp8 -f servidor/casos/c4_baja_logica.sql
  UPDATE producto SET activo = FALSE WHERE nombre = 'Fugazzeta';
$ (instantánea nueva) · cargar_snapshot.sql · semaforo.sql
| a0c40001 | b0c40000 | RECHAZADO | Fugazzeta fue dado de baja |
$ sqlite3 ... < casos/c4_catalogo.sql   -- catálogo: 9 productos, sin la Fugazzeta
| id | nombre    | disponible | eliminado |      PRAGMA foreign_key_check: 0 filas
| 1  | Fugazzeta | 0          | 1         |      items_que_apuntan_a_la_fugazzeta: 1
```

La vista manda la lápida (`eliminado = t`) y la recarga hace `UPSERT`: la fila queda con `eliminado = 1`, el catálogo la filtra y el borrador que la usa sigue cumpliendo su clave foránea.

C5 · Borrador sin cambios → `SIMULADO_APROBADO`

```
$ sqlite3 ... < casos/c5_borrador.sql   -- 1 Napolitana y 2 Cerveza 1L; el servidor no cambia
$ (instantánea nueva) · cargar_snapshot.sql · semaforo.sql
| a0c50001 | b0c50000 | SIMULADO_APROBADO | coincide con la instantánea (simulación: no es una venta) |
```

Todo coincide con la instantánea. Es una simulación: no se creó ningún pedido ni se descontó stock (verificación de abajo).

C6 · Reenviar la misma solicitud → no se duplica y no se reevalúa

```
$ sqlite3 ... < casos/c6_reenvio.sql
solicitudes_antes: 6
-- INSERT con el mismo UUID (a0c50001):
Error near line 6: UNIQUE constraint failed: cola_sync.solicitud_uuid
solicitudes_despues: 6
-- UPDATE para volverla a PENDIENTE:
Error near line 12: La solicitud ya fue evaluada: su estado no cambia
$ (instantánea nueva) · cargar_snapshot.sql · semaforo.sql
| solicitudes_evaluadas_en_esta_corrida | 0 |      -- las 6 solicitudes quedan igual
```

`solicitud_uuid` es la clave primaria (SQLite informa la violación como *UNIQUE constraint failed*). El semáforo solo evalúa `PENDIENTE` y un trigger impide cambiar una solicitud ya evaluada, así que repetirlo no cambia nada. **Parada 8:** en un sistema real la protección va en el servidor: una tabla de solicitudes procesadas con `solicitud_uuid` como clave primaria, insertada en la misma transacción que crea el pedido; si el UUID ya existe, el servidor devuelve el resultado guardado en vez de crear otro pedido.

Verificación final en PostgreSQL: la simulación no cambió pedidos ni stock

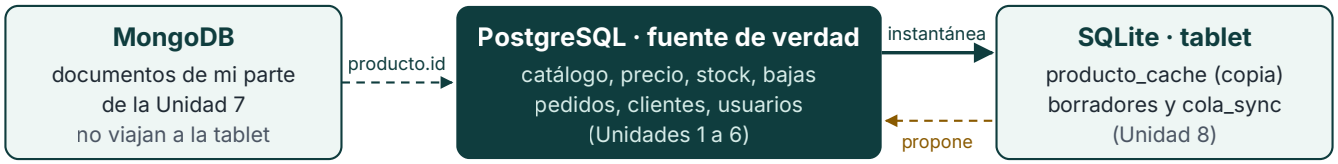
`servidor/verificacion_final.sql` compara la base con la foto que se tomó antes de los casos (`foto_inicial.sql`: productos, pedidos y líneas, fila por fila). Salida en mi PC, resumida (completa en `evidencias/2c_verificacion_servidor.txt`):

	Al empezar	Ahora
Pedidos	7	7
Líneas de `detalle_pedido`	12	12
Productos	12	12
Stock total	620	620

```
Pedidos o líneas que faltan, que cambiaron o que son nuevos (EXCEPT en los dos sentidos): 0 filas
Productos que faltan, que son nuevos o cuyo stock cambió: 0 filas
Lo que sí cambió (2 filas): Fugazzeta activo t → f (C4) · Muzzarella precio 1050.00 → 1150.00 (C2)
```

Hubo tres `SIMULADO_APROBADO` y el servidor sigue con los mismos 7 pedidos, las mismas líneas y el mismo stock. Lo único distinto son los dos cambios que se prepararon a mano en el servidor para C2 y C4: la tablet no escribió nada en PostgreSQL.

3.a Mapa de la capa de datos



v_snapshot_movil → \copy → servidor_snapshot.csv → .import → snapshot_servidor → UPSERT → producto_cache

Vuelta: la tablet solo arma solicitudes (cola_sync). En este TP las evalúa el semáforo contra la instantánea; en un sistema real las recibiría el servidor, que decide precio, stock y venta.

Motor	Qué guarda	Quién manda
PostgreSQL (U1 a U6)	`categoria`, `producto` (precio, stock, baja lógica), `cliente`, `pedido`, `detalle_pedido`, la auditoría de precios y los usuarios.	**Manda sobre todo lo que decide una venta:** si un producto existe, su precio, su stock y su baja; los pedidos y los clientes. Es la fuente de verdad.
MongoDB (mi parte de la U7)	Los documentos semiestructurados de mi parte de la Unidad 7, que referencian productos por el `id` de PostgreSQL.	Manda sobre el contenido de esos documentos. Si el producto que referencian existe, cuánto cuesta y cuánto stock tiene lo dice PostgreSQL. No viajan a la tablet.
SQLite (tablet, U8)	`producto_cache` (copia con fecha), `snapshot_servidor`, `pedido_borrador`, `item_borrador`, `cola_sync`.	No manda sobre ningún dato del catálogo: lo pisa cada instantánea. Solo es dueña de sus borradores y de la cola hasta que el servidor decide. `SIMULADO_APROBADO` no es una venta.

3.b Plan de mantenimiento (agregado al plan de la Unidad 6)

¿Qué se respalda y cómo se prueba la restauración?

- **PostgreSQL**, la fuente de verdad, con el respaldo del plan de la U6 (`backup_foodstore.sh`, `pg_dump -F c`) y su prueba de restauración en una base aparte (`restore_test.sh`). Se agrega un control: desde la base restaurada se exporta `v_snapshot_movil` y se carga en una tablet de prueba con `cargar_snapshot.sql`; tiene que pasar los controles de la carga (entre ellos, que lleguen todas las filas de la vista) y el semáforo tiene que correr. Si no, el respaldo no sirve para la feria.
- **MongoDB** con `mongodump`, restaurado y comprobado de la misma forma, en una base aparte.
- **La tablet no se respalda como fuente:** el catálogo se reconstruye con una instantánea. Lo único que existe solo en la tablet son los borradores y las solicitudes sin enviar; al cerrar cada feria se copia el archivo con `sqlite3 foodstore_movil.db ".backup feria.db"` y se controla con `PRAGMA integrity_check`.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?

- Nada por su cuenta: el servidor sigue mandando aunque haya vuelto atrás. Las tablets **no corrigen el servidor con su copia**, aunque tengan precios más nuevos.
- Se publica una instantánea nueva desde la base restaurada y las tablets la cargan antes de seguir: el `UPSERT` pisa precio y stock; los productos creados después del respaldo no vienen y quedan como lápidas, así que los borradores que los usan salen `RECHAZADO` («no existe en el servidor»).
- Las solicitudes que el servidor procesó después del respaldo se perdieron con él: las tablets las reenvían con el mismo UUID y la tabla de solicitudes procesadas del servidor (parada 8) evita duplicar las que sí quedaron.

¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO?

- **Al cierre de cada jornada de feria**, el encargado del food truck revisa en la tablet las `CONFLICTO` y `RECHAZADO` con su motivo. Un conflicto se resuelve con el cliente (acepta el precio y sale una solicitud nueva, o se anula el borrador); un rechazo se anula o se rearma con otro producto.
- **Una vez por semana**, junto con el health check de la U6, se cuentan por motivo. Si los conflictos de precio son muchos, la instantánea se exporta más seguido, por ejemplo antes de cada feria.

3.c Code review con IA

Se lo pedí a **OpenCode, agente Plan** (modelo Big Pickle), abierto en la raíz del repo `tpi-food-store`: revisar todos los `.sql` y `.sh` de `sql/` (TPI, Unidades 1 a 3), `trabajos-practicos/unidad1/` (S2 = Semana 2) y este TP, sin leer informes ni evidencias y sin modificar nada. Devolvió 34 hallazgos (conversación en `docs/code_review/revision_opencode.md`). Antes de decidir, cada uno se comprobó en la base o leyendo el script; `docs/code_review/comprobar_hallazgos.sh` repite las comprobaciones sobre copias y deja la salida en `evidencias/3c_code_review.txt`. **16 aceptados** (el problema es real y queda para la entrega final del TPI, porque el script es de una unidad ya entregada), **7 corregidos** en este TP y **11 descartados**.

#	Unidad · archivo	Hallazgo de OpenCode	Decisión	Motivo
1	U2 · `sql/07` (7.2), `05b`	La facturación filtra `p.activo`: dar de baja un producto borra sus ventas pasadas del reporte	`Aceptado`	Comprobado (desarrollado abajo). Contradice `09_soft_delete.sql`.
2	U1 · `sql/01_schema.sql`	Los `CASCADE` borran vistas, la materializada y el procedimiento sin avisar	`Aceptado`	Comprobado: tras los `DROP TABLE` queda 1 vista de 5 y 0 materializadas; el `DROP TYPE` se lleva `sp_registrar_pedido`.
3	U1 · `sql/03`, `sql/06`	El trigger de stock valida pero no descuenta; con dos ventas a la vez la segunda falla con un error poco claro	`Aceptado`	Comprobado (desarrollado abajo).
4	U8 · `movil/cargar_snapshot.sql`	Un CSV cortado deja como lápidas a todos los productos que faltan	`Corregido`	Comprobado (desarrollado abajo).
5	U8 · `movil/semaforo.sql`	Dos solicitudes que juntas superan el stock salen las dos en verde	`Aceptado`	Límite de la simulación (2 × 10 flanes con 15: dos verdes). Lo frena el servidor al confirmar (#3). Documentado en el script.
6	U1 · `sql/03b`	`LIMIT 1` sin `ORDER BY` para elegir el pedido de prueba	`Aceptado`	Hoy no falla (el pedido 1 no tiene esos productos); es un riesgo latente.
7	U2 · `sql/02b`	Asume IDs contiguos y no es idempotente	`Aceptado`	Funciona porque el TPI recrea la base; se documenta que no se corre dos veces.
8	U2 · `sql/07` a `11`	IDs fijos que solo existen después de la carga masiva	`Aceptado`	Dependen del orden de `ejecutar_todo.sh`; se documenta.
9	U1 · `crear_base.sh` (S2)	Con `{ … } | tee` el código de salida es el de `tee`	`Aceptado`	Comprobado: sin `pipefail` da 0 aunque el bloque falle; con `pipefail`, 1.
10	U1 · `restricciones.sql` (S2)	La Regla A deja cambiar el producto de una línea con el precio congelado	`Aceptado`	Comprobado: Muzzarella → Napolitana conserva 1000 con precio de lista 1500.
11	U1 · `sql/10_auditoria_precios.sql`	`DROP TABLE` borra el historial al volver a correrlo	`Aceptado`	Real sobre una base con datos; en el TPI la base se recrea entera.
12	U3 · `sql/05b`, `11`	La vista materializada no se refresca antes de medir	`Aceptado`	`08` confirma un pedido después de crearla: falta un `REFRESH`.
13	U8 · `servidor/v_snapshot_movil.sql`	`ROUND(precio * 100)::INTEGER` desborda con un precio alto	`Corregido`	Pasa a `BIGINT`. Comprobado: `INTEGER` da «entero fuera de rango»; `BIGINT` devuelve 2500000000.
14	U8 · `movil/schema_sqlite.sql`	Faltan controles: evaluada sin motivo, `disponible` con `eliminado`, formato de `copiado_en`	`Corregido`	Lo corrigió OpenCode (Build) en su esquema, a pedido: tres `CHECK`.
15	U1 · `sql/08_transacciones.sql`	Tiene errores intencionales y cambios confirmados	`Descartado`	Los errores son la prueba de atomicidad; corre sin `ON_ERROR_STOP` sobre una base que se recrea.

#	Unidad · archivo	Hallazgo de OpenCode	Decisión	Motivo
16	U1 · `lectura_critica.sh` (S2)	No revisa el código de salida de `psql`	Aceptado	Como #9: puede terminar «bien» con una evidencia incompleta.
17	U1 · `laboratorio_concurrencia.sh` (S2)	La restauración falla con la tabla vacía	Aceptado	Con `MAX(id)` en `NULL`, el `DELETE … WHERE id > $maximo` falla.
18	U8 · `servidor/verificacion_final.sql`	No ve un producto nuevo ni un cambio que no mueve los totales	Corregido	Foto fila por fila y `EXCEPT` en los dos sentidos. Comprobado: la versión anterior no veía un producto agregado.
19	U8 · `movil/pruebas_sqlite.sql`	La solicitud de 1.c queda `PENDIENTE` hasta C2	Descartado	La pide la consigna y no cambia ningún caso: cada uno tiene su solicitud.
20	U8 · `servidor/schema.sql`	El servidor del TP8 no tiene las reglas A, B y C de la Semana 2	Descartado	Comprobado (desarrollado abajo).
21	U1 · `restricciones.sql` (S2)	La Regla B no controla un `INSERT` en una categoría inactiva	Aceptado	Solo mira el `UPDATE` de la categoría.
22	U8 · `movil/cargar_snapshot.sql`	`.import` es del CLI y no rechaza columnas extra	Descartado	`sqlite3` es la herramienta de la consigna (el README explica DB Browser). Ignorar columnas extra es a propósito: se copian por nombre.
23	U8 · `movil/casos/c2_acepta.sql`	Actualiza el precio visto de todos los ítems	Descartado	El cliente acepta los precios del motivo; los que no cambiaron quedan igual.
24	U8 · `movil/semaforo.sql`	Un motivo podría quedar en `NULL`	Descartado	Solo con las claves foráneas apagadas; con #14 tampoco se puede guardar.
25	U1 · esquemas del TPI y del TP1	Los controles de email no son iguales	Aceptado	Queda para unificar en la entrega final.
26	U8 · `servidor/v_snapshot_movil.sql`	`eliminado` no mira la categoría	Descartado	Comprobado: con Bebidas inactiva, la Coca viaja con `disponible = f` y se rechaza igual.
27	U8 · `movil/schema_sqlite.sql`	`item_borrador` sin `ON DELETE CASCADE`	Descartado	Borrar borradores no es parte del flujo; el `RESTRICT` protege el historial.
28	U8 · `comprobar_hallazgos.sh`	Algunas pruebas no muestran lo que dice el título	Corregido	Rehecho con esta numeración; #1 elige un producto activo y #2 separa `DROP TABLE` de `DROP TYPE`.
29	U8 · `comparar_versiones.sh`	No controla que exista `version_claude/semaforo.sql`	Corregido	Se agregó el control, y corta si no puede copiar los archivos.
30	U2 · `sql/02c`	Define «recientes» por ID y asume que se corrió `02b`	Aceptado	Como #7 y #8.
31	U1 · `data.sql` (S2)	`row_number() OVER (ORDER BY fecha)` sería ambiguo con fechas repetidas	Descartado	Comprobado: ninguna fecha se repite (0 filas).
32	U8 · `movil/schema_sqlite.sql`	`abs(random())` puede desbordar	Corregido	Lo corrigió OpenCode: `random() & 3`.
33	U1 · `sql/04_indices.sql`	No crea ningún índice	Descartado	A propósito, y el archivo lo dice: los índices están en `01` y se miden en `11`.
34	U2 · `sql/extra/carga_masiva_original.sql`	Es lenta y no hace `ANALYZE`	Descartado	Versión original guardada para compararla en la Semana 3; no es parte del flujo.

> **#3 · Unidad 1 · Aceptado. Cómo se comprobó:** sobre una copia de la base del TPI. Primero, dentro de `BEGIN … ROLLBACK`, un `UPDATE` de una línea ya vendida (cantidad 2 → 3): el stock quedó en 12, porque el trigger solo valida y el descuento está en `sp_registrar_pedido`. Después, con dos sesiones reales de `psql`: la A pide las 8 unidades de Queso cremoso 500g dentro de una transacción y espera 2 s antes del `COMMIT`; la B pide las mismas 8 medio segundo después. **Resultado:** la B queda esperando el bloqueo de fila; cuando la A confirma, el stock daría −8 y la B falla con *viola la restricción «check» «producto_stock_check»*. Stock final 0 y un solo pedido nuevo: el stock no queda negativo, pero el mensaje no le dice nada al usuario y un `UPDATE` de cantidad lo desincroniza. Para la entrega final: que el trigger descuente `NEW.cantidad − OLD.cantidad` o que las ventas pasen solo por el procedimiento, con un mensaje claro.

> **#1 · Unidad 2 · Aceptado. Cómo se comprobó:** dentro de `BEGIN … ROLLBACK` se calculó la consulta 7.2, se dio de baja el producto activo con más ventas (Queso cremoso 500g, Lácteos) y se volvió a calcular. **Resultado:** Lácteos bajó de 100.254.249,64 a 100.049.049,64: desaparecieron 205.200,00 de ventas, aunque las 9 líneas siguen en `detalle_pedido`. El filtro de `activo` va en el catálogo, no en un reporte de ventas pasadas.

> **#4 · Unidad 8 · Corregido. Cómo se comprobó:** se cargó en una tablet de prueba un CSV cortado, con 5 de las 12 filas. **Resultado:** la versión anterior lo cargaba sin error y dejaba 7 lápidas: la tablet creía que 6 productos vigentes estaban dados de baja. Corrección: la vista manda `filas_totales = COUNT(*) OVER ()` y la carga, antes de tocar `snapshot_servidor`, exige en una tabla de control que lleguen todas las filas de una sola copia. Con el mismo CSV ahora falla con *CHECK constraint failed: completa = 1*, y como corre entre `BEGIN` y `COMMIT` con `.bail on`, el catálogo queda como estaba (1 lápida, la de antes).

> **#20 · Unidad 8 · Descartado.** OpenCode marcó que el servidor del TP8 no tiene las reglas A, B y C de la Semana 2. **Cómo se comprobó:** se armó otra copia con esas reglas (sus tres triggers), se corrieron ahí las preparaciones de C2 y C4 y se comparó `v_snapshot_movil` con la de `foodstore_tp8`. **Resultado:** las dos preparaciones pasan y la instantánea es idéntica. La tablet nunca escribe en el servidor, así que esas reglas no cambian nada de lo que se prueba en este TP.

Antes de esta revisión, Claude había encontrado y corregido que `cargar_snapshot.sql` llenaba la instantánea por posición (bitácora, entrada 1).